Abstract class and Interfaces

Abstract Classes

- Sometimes we might want to declare a class whose full implementation is not known
- Allows declaring the method without writing any code for it in that class
- If a method is declared abstract, then the class must also be declared as abstract
- private or static method cannot be declared abstract because,
  - a private method cannot be accessed outside and
  - all static methods are implicitly final, they cannot be overridden

Abstract Example

```
abstract class Mammal
{
   abstract void a();
}

public class Human extends Mammal
{
   public void a()
    {
    // body comes here
    }
}
```

```
public class GangMember extends
 Mammal
 {
   public void a()
   {
       //  definition
 } }

 public class Dog extends Mammal
 {
   public void a()
    {
   // definition
     }
 }
```

Implementing Interfaces

- An abstract class will have some methods that are declared as abstract and some that are not
- If an entire class is to be declared abstract, interface is the best choice
- An interface is an entirely abstract class
- Interface can also be derived in a manner as deriving a subclass from another class

Interface Example

```
interface Clock {
    public String GetTime(int hour);  }
class Cuckoo implements Clock  {
    public String GetTime(int hour) {
        StringBuffer str = new StringBuffer();
        for (int i=0; i < hour; i++)
            str.append("Cuckoo ");
        return str.toString();
  }  }
class Watch implements Clock  {
        public String GetTime(int hour) {
            return new String("It is " + hour + ":00");
  }  }
```

Type of Errors: 3 types

Syntax errors: rules of lang not followed. They are detected by the compiler

- Misspelled variable and function names,
- Missing semicolons,
- Improperly matches parentheses and curly braces
- Incorrect format in selection and loop statements

R*untime errors:* occur while program is running and if the env. detects an operation that is impossible to carry out

- divide by zero error
- trying to open a file that doesn't exist

Type of Errors: 3 types

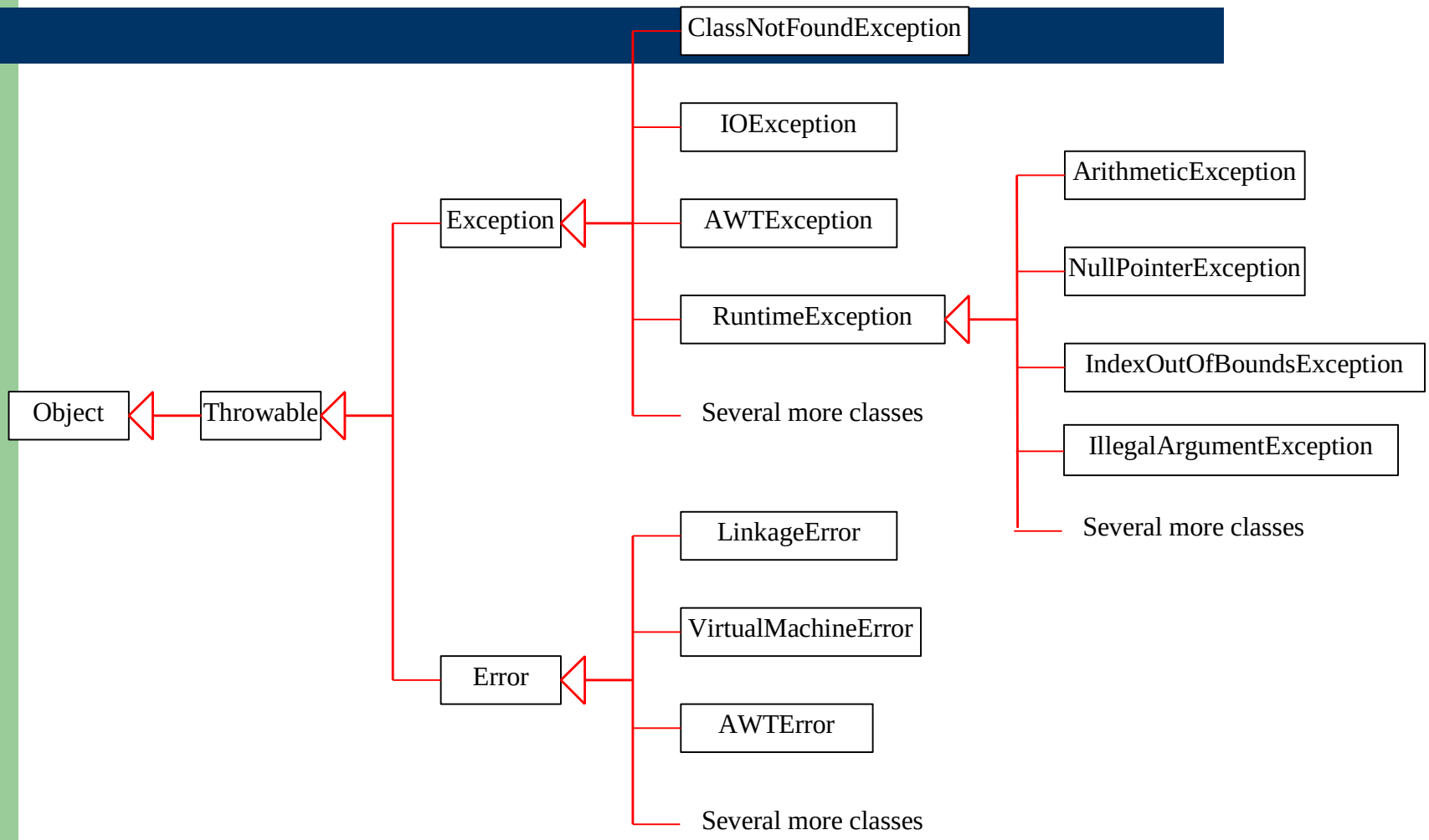
Logical errors: occur when a program doesn't perform the way it was intended to

- Multiplying when you should be dividing
- Adding when you should be subtracting
- Opening and using data from the wrong file
- Displaying the wrong message

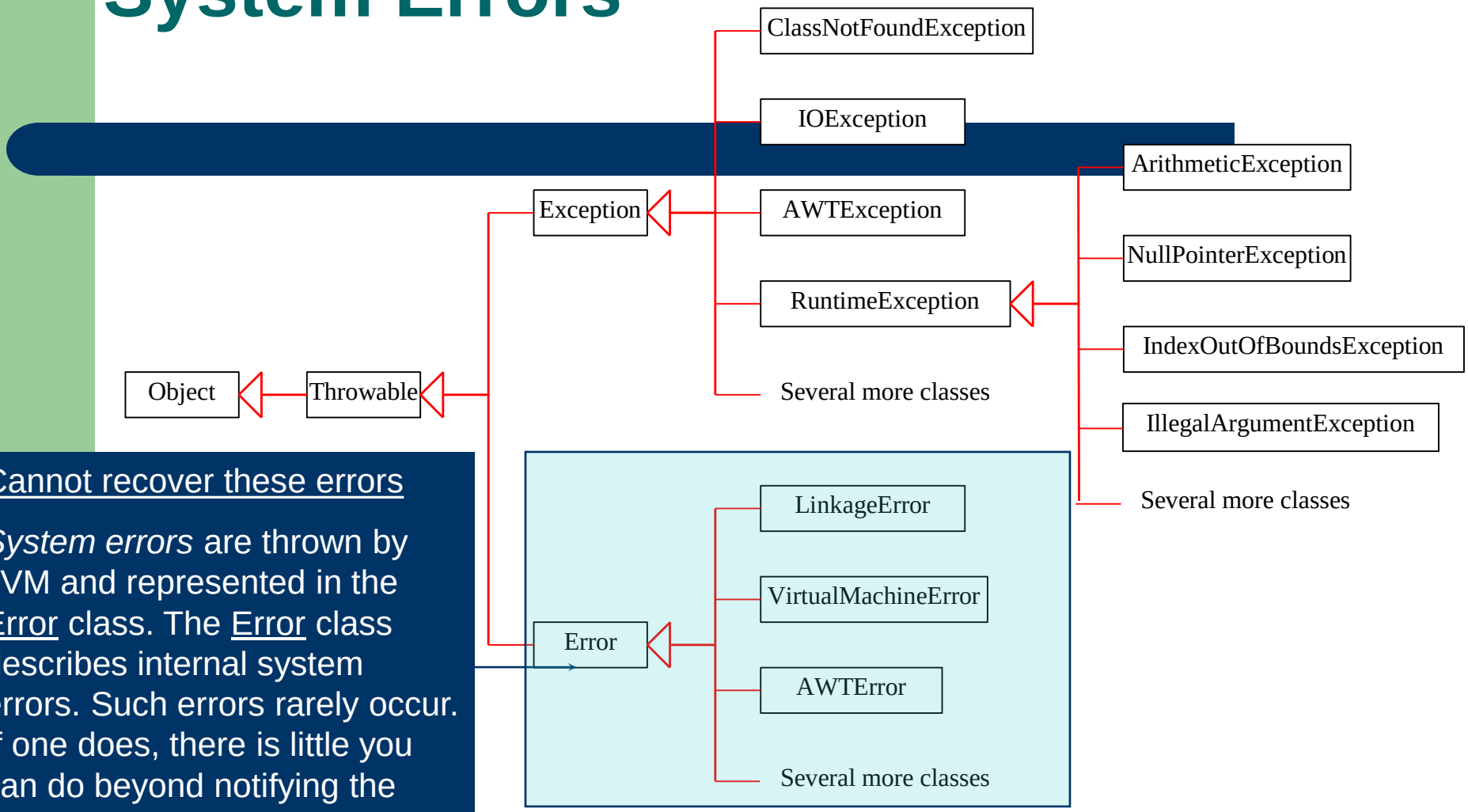
What Is an Exception?

- The term *exception* is shorthand for the phrase "exceptional event.“
- **Definition:** An *exception* is an event, which occurs during the execution of a program, that disrupts the normal flow of the program's instructions.

Exception Classes



System Errors

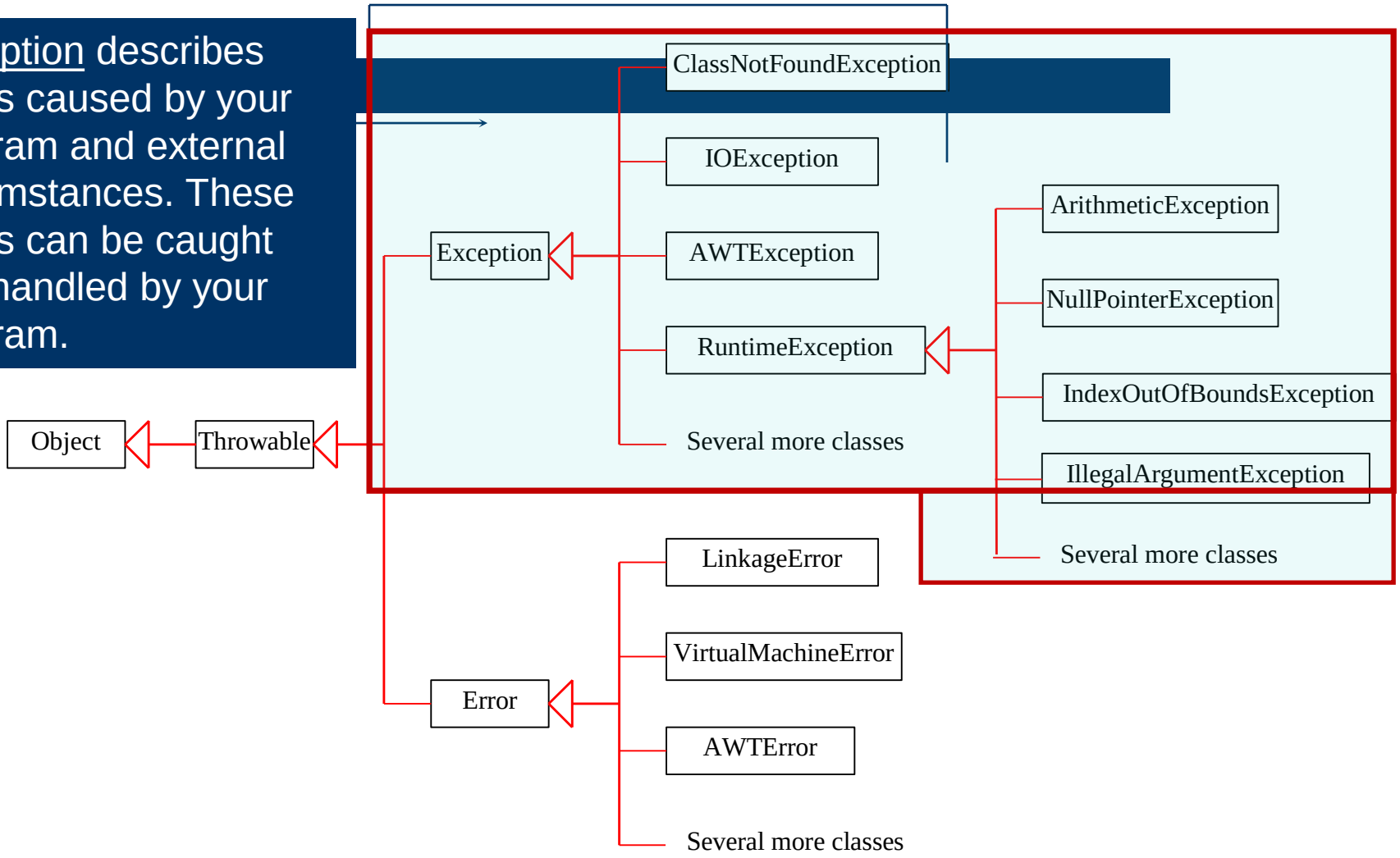


Cannot recover these errors

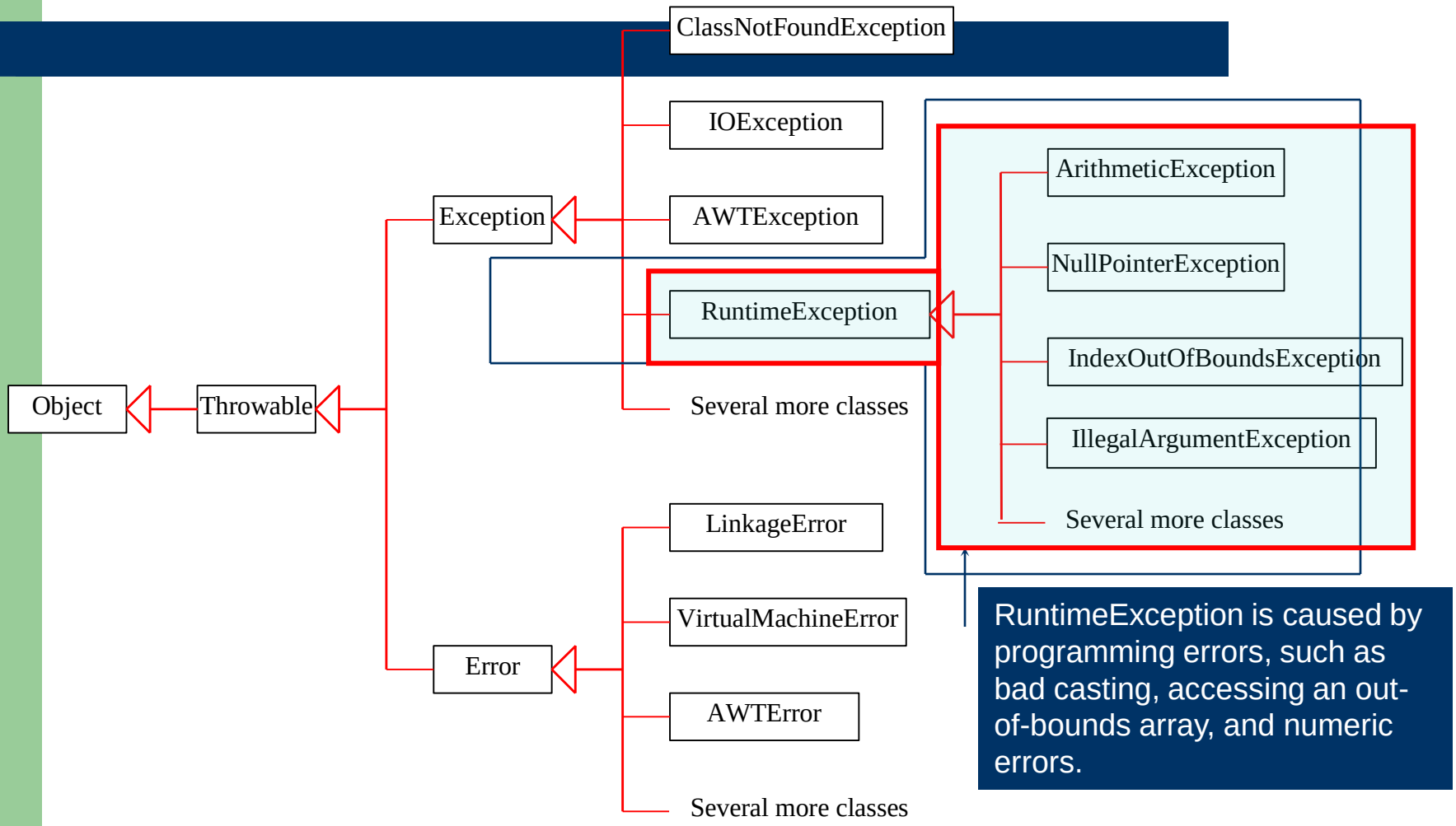
System errors are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

Exceptions

Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.



Runtime Exceptions



Checked Exceptions vs. Unchecked Exceptions

RuntimeException, Error and their subclasses are known as *unchecked exceptions*.

Ex: bad data provided by user during the user-program interaction

All other exceptions are known as *checked exceptions*, meaning that the compiler forces the programmer to check and deal with the exceptions.

These exceptions are ***checked*** at Compile time.

Eliminating Software Errors: Exception handling

- Exception in Java is abnormal state at runtime
- Whenever an exception occurs, the program code should handle it, else the program execution is terminated by the runtime system
- Java's exception handling makes use of 5 keywords - try, catch, throw, throws and finally

logic

- When an error occurs within a method,
- the method creates an object and hands it off to the runtime system.
- The object, called an *exception object*, contains information about the error (How?), including its type and the state of the program when the error occurred.
- Creating an exception object and handing it to the runtime system is called *throwing an exception*.